

树与生成树

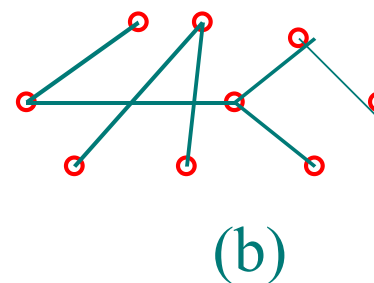
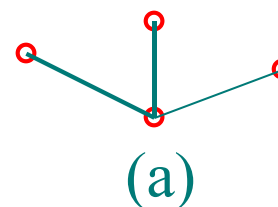
一.树 (Tree)

1.树的定义:一个连通无回路的无向图 T ,称之为树.

2.树叶:度数为1的结点,称为树叶.

3.分支结点(内结点):度数大于1的结点.

4.森林:一个无向图的每个连通分支都是树.



5.与树定义等价的几个命题

定理1 给定图 T , 以下关于树的定义是等价的.

- (1) T 无回路的连通图.
- (2) T 无环且每对结点之间有一条且仅有一条道路.
- (3) T 无回路但在任一对不相邻的顶点间添加一条新边 e , 则 $T+e$ 包含唯一的圈.
- (4) T 连通的, 且每条边都是割边.
- (5) T 连通的且 $m=n-1$.
- (6) T 无回路且 $m=n-1$.

证明: (1) $\Rightarrow$ (2): 已知 T 是连通无回路图, 所以 T 中无环。
若 T 中存在两条不同的 $u-v$ 道路 P_1 和 P_2 , 则存在 P_1 的一条边 $e=xy$, 它不是 P_2 的边, 因此 x, y 在 $(P_1 \cup P_2) - e$ 连通, 所以, $(P_1 \cup P_2) - e$ 存在一条 $x-y$ 路 P , 故 $P+e$ 是 T 的回路, 矛盾。

(2) T 无环且每对结点之间有一条且仅有一条道路.

(3) T 无回路但在任一对不相邻的顶点间添加一条新边 e , 则 $T+e$ 包含唯一的圈.

(4) T 连通的, 且每条边都是割边.

(2) $\Rightarrow$ (3): 显然 T 无回路, 否则对回路上的任一对顶点都至少存在两条路, 与(2)矛盾. 设 u, v 是 T 中任意两个不相邻的点, 令 $e=uv$, 由(2), T 中有一条唯一的 $u-v$ 道路, 所以 $T+e$ 中包含唯一的圈。

(3) $\Rightarrow$ (4): 因为 T 无回路, 所以 T 的每条边都是割边; 若 T 不连通, 设 T_1, T_2 是 T 的两个分支, 设 $u \in V(T_1), v \in V(T_2)$, 则 $uv \notin E(T)$, 显然 $T+uv$ 不存在回路, 与(3)矛盾。

(4) T 连通的,且每条边都是割边.

(5) T 连通的且 $m=n-1$.

(4) $\Rightarrow$ (5):关于点数用归纳法证明。

当 $n=1$ 或 2 时, T 是平凡图或 K_2 , 显然有 $m=n-1$ 。

假设 $n \leq k$ 时结论成立, 往证 $n=k+1$ 时成立。

当 $n=k+1$ 时。取 T 的一条边 e , 由(4), e 是割边,

所以 $T-e$ 有两个分支 T_1 和 T_2 ,

因为 $|V(T_1)| \leq k$, $|V(T_2)| \leq k$,

所以, 由归纳假设, 有

$$|E(T_1)| = |V(T_1)| - 1, \quad |E(T_2)| = |V(T_2)| - 1$$

$$\text{故 } m = |E(T_1)| + |E(T_2)| + 1$$

$$= |V(T_1)| - 1 + |V(T_2)| - 1 + 1$$

$$= n - 1。$$

(5) T 连通的且 $m=n-1$.

(1) T 无回路的连通图.

- ❖ (5) $\Rightarrow$ (1): 只需证明 T 无回路。对 T 关于顶点数归纳。
- ❖ 当 $n=1$ 或 $n=2$ 时，显然成立。
- ❖ 假设 $n \leq k$ 时结论成立，往证 $n=k+1$ 时成立。
- ❖ 因为 T 连通，所以 $\delta(T) \geq 1$ ，由 $m=n-1$ 及 $\sum d(v)=2m$ 得， T 中至少存在一点 u ，使得 $d(u)=1$ 。考虑 $T'=T-u$ ，显然 T' 连通，且 $|E(T')|=|V(T')|-1$ ，由归纳假设， T' 无回路，所以 T 无回路。

(5) T 连通的且 $m=n-1$.

(6) T 无回路且 $m=n-1$.

❖ (5) $\Rightarrow$ (6): 只需证明 T 无回路。对 T 关于顶点数归纳。

❖ 当 $n=1$ 或 $n=2$ 时，显然成立。

❖ 假设 $n \leq k$ 时结论成立，往证 $n=k+1$ 时成立。

❖ 因为 T 连通，所以 $\delta(T) \geq 1$ ，由 $m=n-1$ 及 $\sum d(v) = 2m$ 得， T 中至少存在一点 u ，使得 $d(u)=1$ 。考虑 $T'=T-u$ ，显然 T' 连通，且 $|E(T')|=|V(T')|-1$ ，由归纳假设， T' 无回路，所以 T 无回路。

(6) T 无回路且 $m=n-1$.

(1) T 无回路的连通图.

(6) $\Rightarrow$ (1):假设 T 不连通, 设 $T_1, T_2, \dots, T_k$ 为 T 的连通分支, 则 $k \geq 2$ 。由(6)得, 对任意的 T_i , T_i 是无回路的连通图, 所以 T_i 是树,

$$|E(T_i)| = |V(T_i)| - 1$$

$$\begin{aligned} \text{故 } m &= \sum_{i=1}^k |E(T_i)| = \sum_{i=1}^k |V(T_i)| - k \\ &= n - k < n - 1 \end{aligned}$$

矛盾。

定理2: 每一非平凡树至少有两片树叶。

证明: 设 T 是一非平凡树, 则 $m=n-1$ 。

因为 T 连通且非平凡, 所以 $\delta(T) \geq 1$ 。

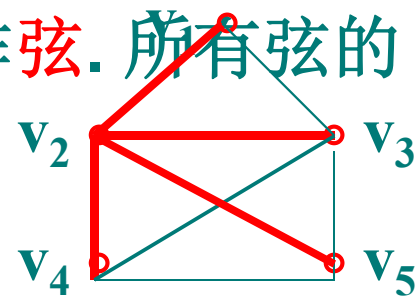
设 T 有 k 片树叶, 则剩余的 $n-k$ 个顶点的度至少为2。由 $\sum d(v) = 2m$ 得, $k + 2(n-k) \leq m = 2(n-1)$,
所以 $k \geq 2$ 。

二. 生成树

1.定义:如果图 G 的生成子图是树,则称此树为 G 的生成树.

2.弦:图 G 中,不在其生成树里的边,称作弦. 所有弦的集合,称为该生成树的补.

定理2 连通图 G 中至少有一棵生成树.



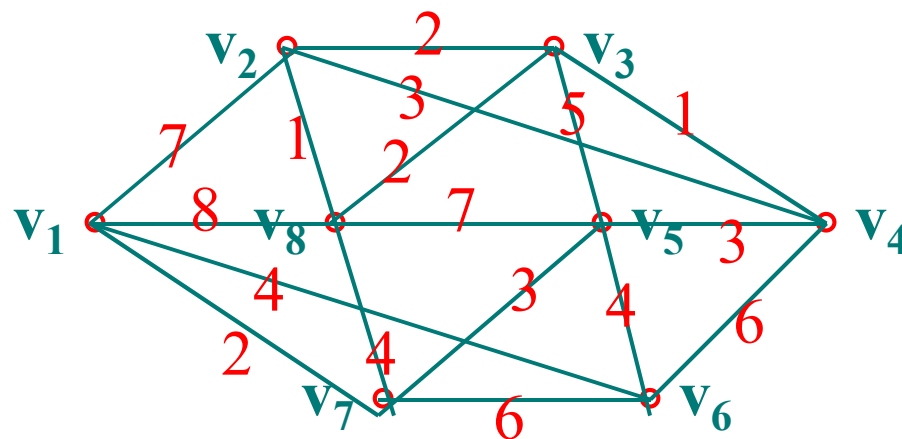
证明:如果 G 中无回路,则 G 本身就是树.

如果 G 中有回路,可以通过反复删去回路中的边,使之既无回路,又连通.就得到生成树.

思考题:设 G 是有 n 个结点, m 条边的连通图,问要删去多少条边,才得到一棵生成树?

三.赋权图的最小生成树

1.定义:一棵生成树中的所有边的权之和称为该生成树的权. 具有最小权的生成树,称为最小生成树.

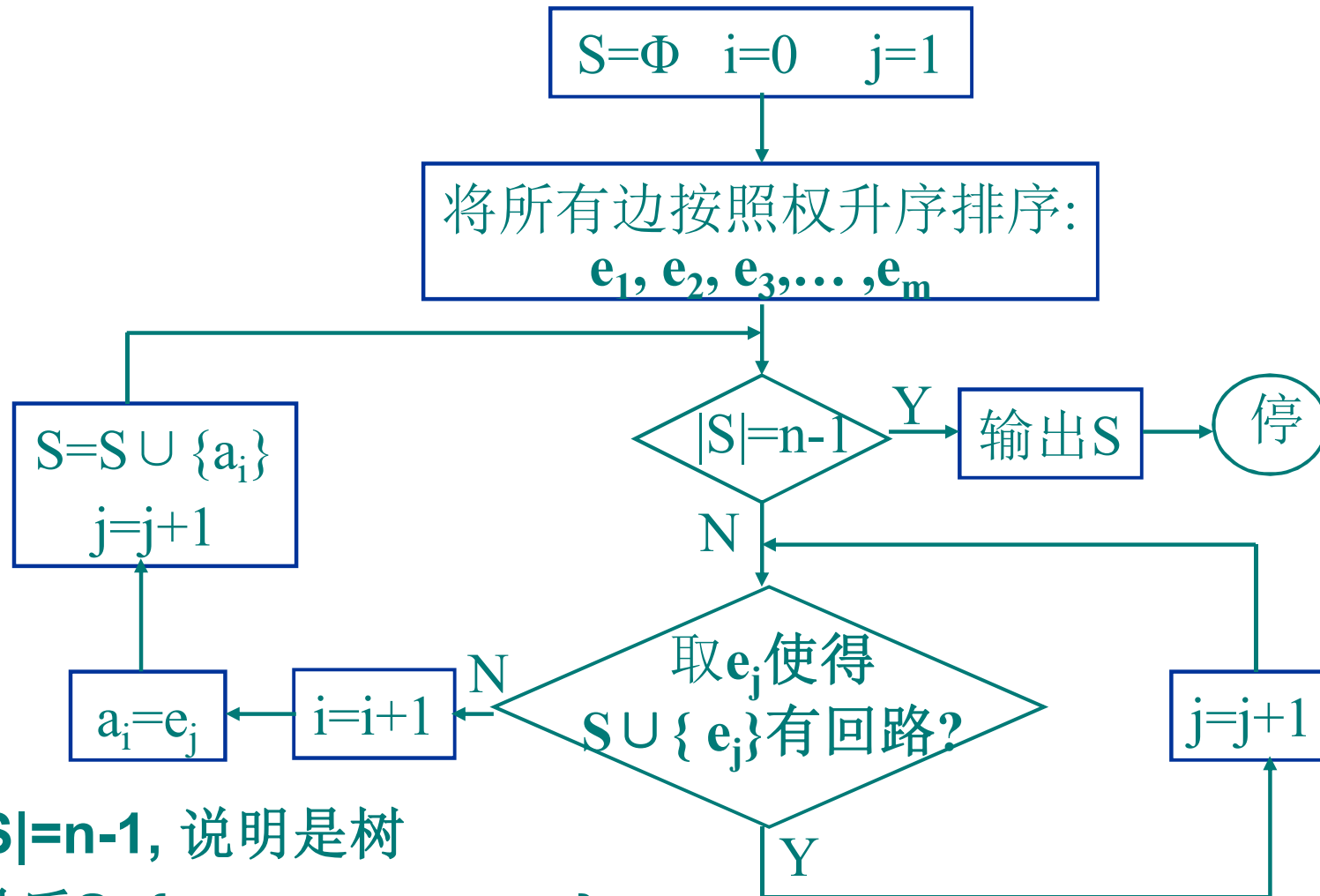


2.求最小生成树算法

---Kruskal算法:

(贪婪算法)

Kruskal算法: 设 G 是有 n 个结点, m 条边($m \geq n-1$)的连通图.

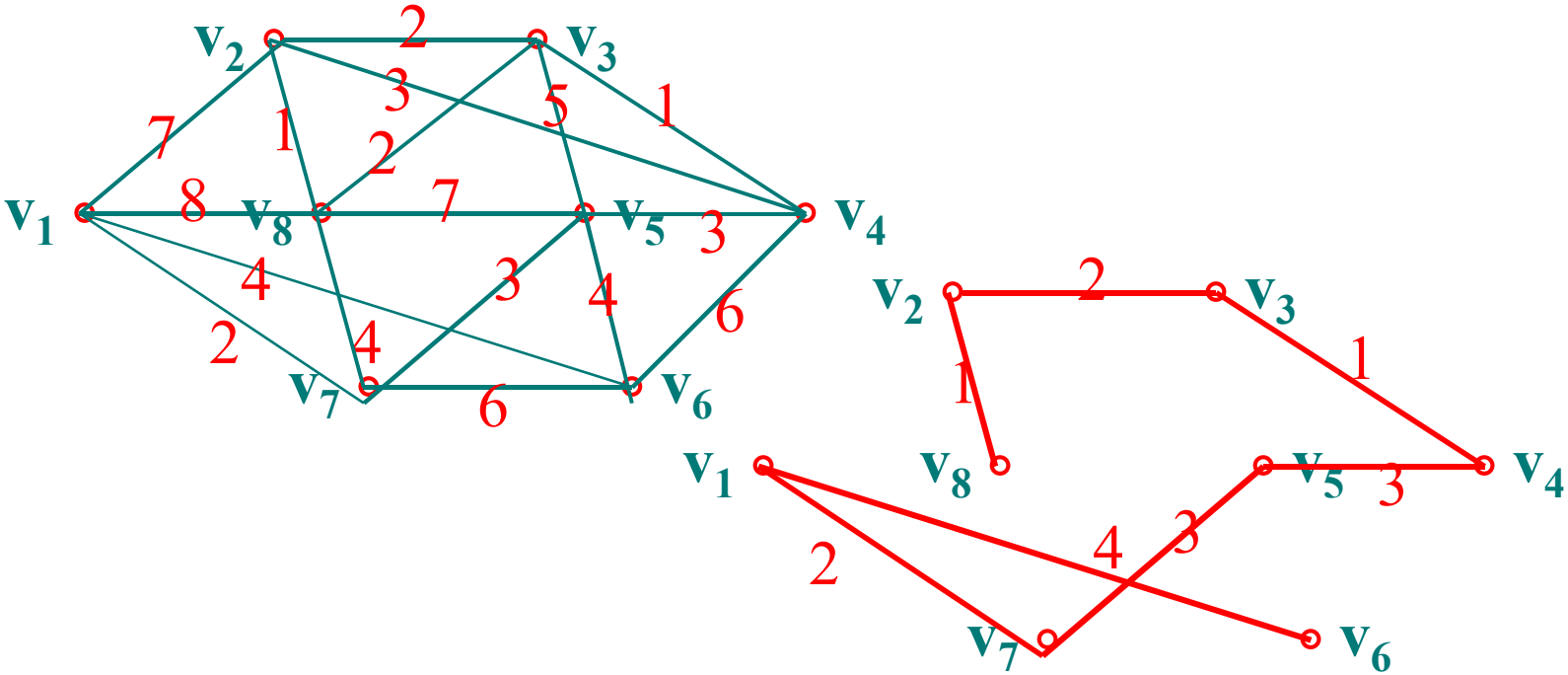


$|S| = n - 1$, 说明是树

最后 $S = \{a_1, a_2, a_3, \dots, a_{n-1}\}$

边按升序排序:边 (v_i, v_j) 记成 e_{ij}

边	e_{28}	e_{34}	e_{23}	e_{38}	e_{17}	e_{24}	e_{45}	e_{57}	e_{16}
权	1	1	2	2	2	3	3	3	4
边	e_{78}	e_{56}	e_{35}	e_{46}	e_{67}	e_{58}	e_{12}	e_{18}	
权	4	4	5	6	6	7	7	8	



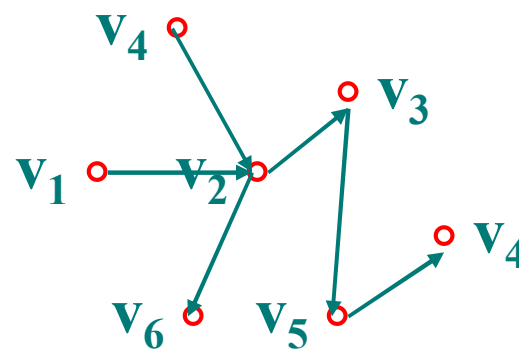
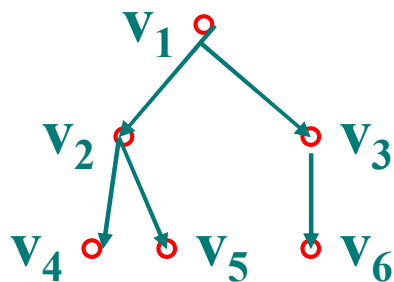
- ❖ 定理1.由Kruskal算法得到的生成树是最小生成树。
- ❖ 证明：设Kruskal算法得到的 G 的生成树为
- ❖ $T^* = \{e_1, e_2, \dots, e_{n-1}\}$
- ❖ 假设 T^* 不是 G 的最小生成树。对 G 的任意一棵生成树 T ,
定义. $f(T) = \min\{i | e_i \notin E(T)\}$
取 G 的最小生成树 T , 使得 $f(T)$ 尽可能大, 设 $f(T) = k$
则 $T + e_k$ 包含唯一的一个圈 C , 且 C 中存在边 e , 使得 e 在 T
中, 但不在 T^* 中,
- ❖ 令 $T' = (T - e) \cup e_k$, 则 T' 也是 G 的生成树, 且
- ❖ T' 的权 = T 的权 + e_k 的权 - e 的权
- ❖ 由Kruskal算法, e 的权 $\geq e_k$ 的权, 故 T' 的权 $\leq T$ 的权
- ❖ 所以 T' 也是 G 的最小生成树, 但 $f(T') > f(T)$, 矛盾

根树及其应用

一.有向树

1.定义:如果 G 是个有向图,且若不考虑边的方向时(即看成无向图时),是一棵树,则称 G 是有向树.

例如:



二.根树:如果一棵有向树,恰有一个结点的入度为0,其余所有结点的入度均为1,则称此树为根树.

1.树根:入度为0的结点.

2.叶:出度为0的结点.

3.分支结点(内结点):出度不为0的结点.

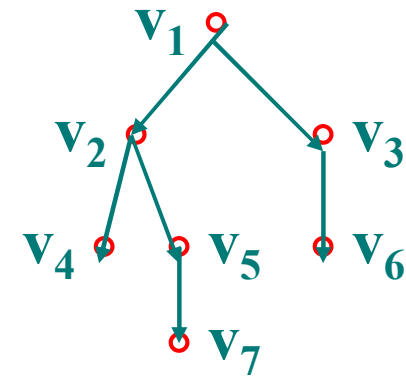
4.父结点与子结点:如果 $\langle v_i, v_j \rangle$ 是根树中

的一条边,则称 v_i 是 v_j 的父结点, v_j 是 v_i 的子结点.

5.祖先结点与后裔结点: 在根树中,如果从 v_i 到 v_j 有路,则称 v_i 是 v_j 的祖先结点, v_j 是 v_i 的后裔结点.

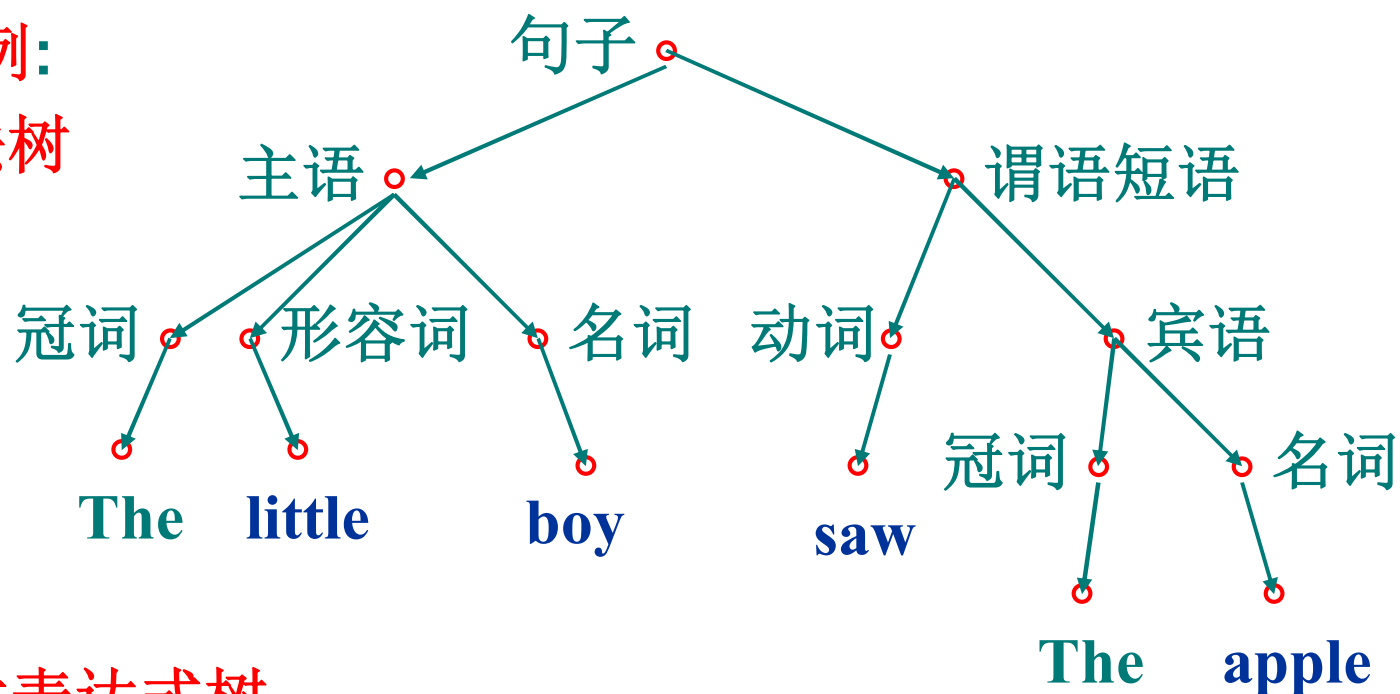
6.根树结点的层次:从根结点到某个结点的路径的长度,称为该结点的层次. 同一层次的结点称为**兄弟结点**.

7.树高:从树根到各个叶结点的路径中, 最长路径的长度, 称为该树的高度(树高).



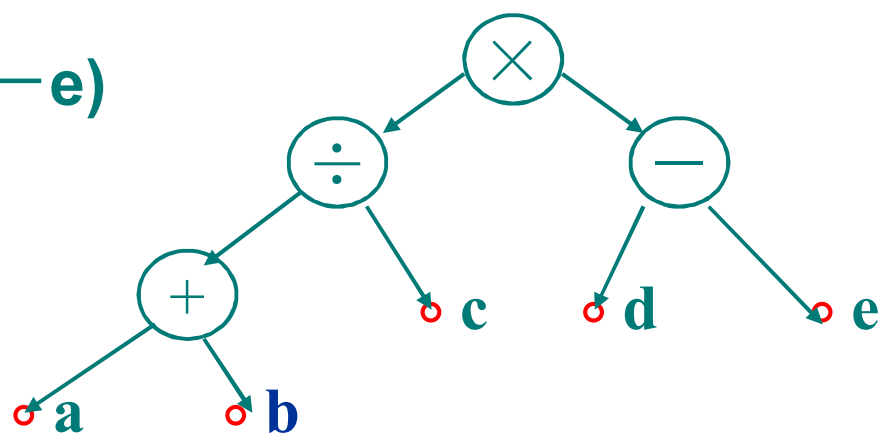
三.举例:

a) 语法树

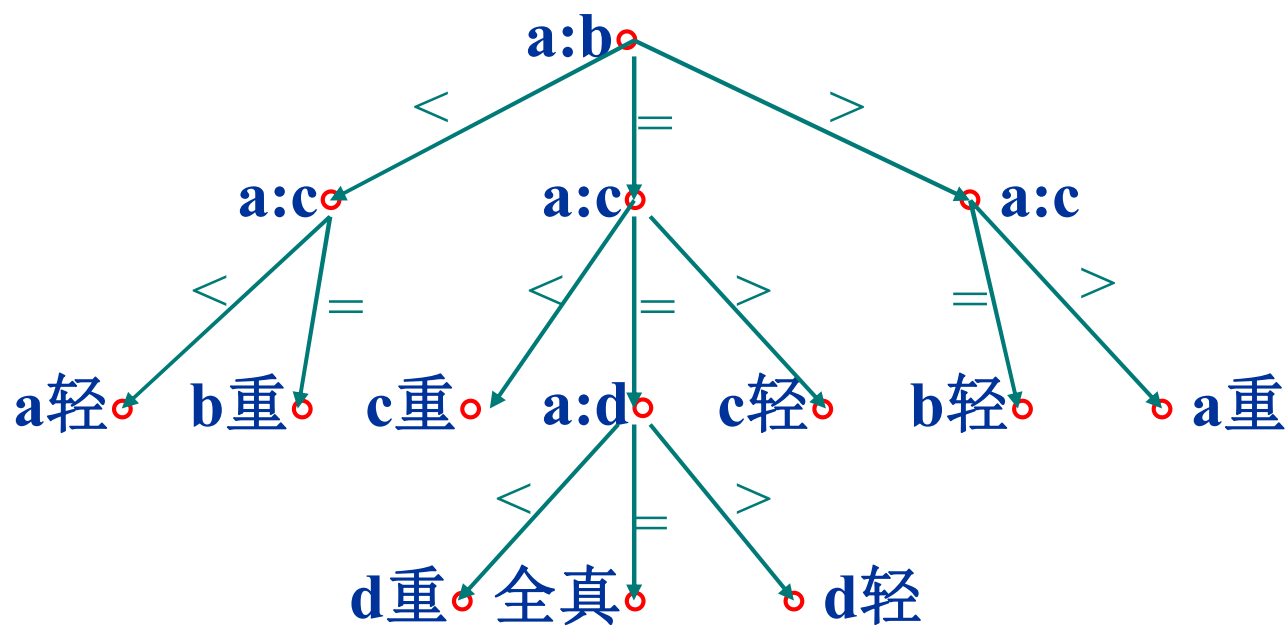


b) 算术表达式树

$((a+b) \div c) \times (d-e)$



c)判定树:有四枚金币**a,b,c,d**,已知道三个是真的,最多一个是假的,它们的外表完全相同,只是重量有点差别.给你一架天平找出假币.

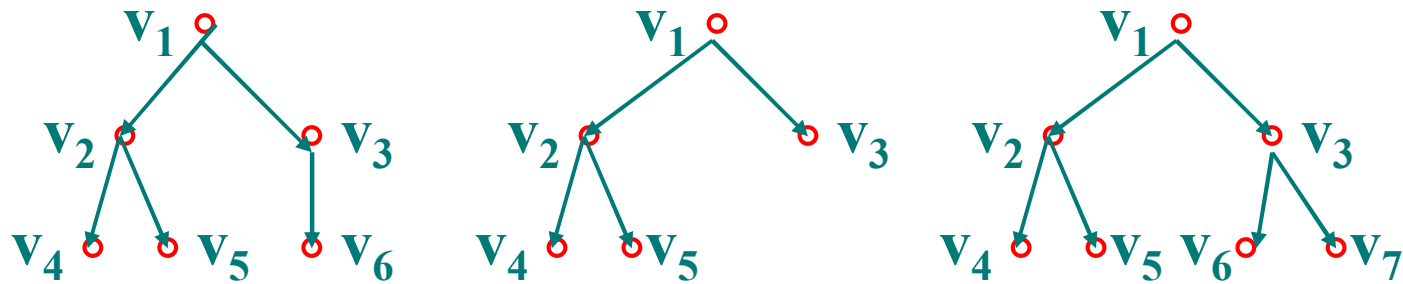


五.m叉树与完全m叉树

1.m叉树:在根树中,如果每个结点的出度最大是m, 则称此树是m叉树.

2.完全m叉树:在根树中,如果每个结点的出度都是m或者等于0, 则称此树是完全m叉树.

3. 正则m叉树:在完全m叉树中,如果所有树叶的层次相同, 则称之为正则m叉树.



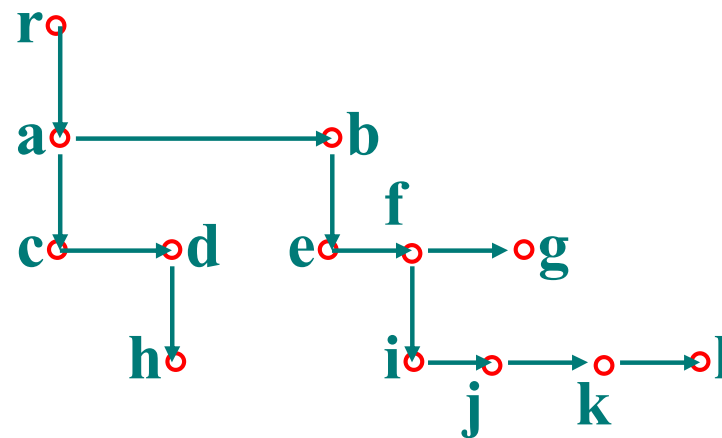
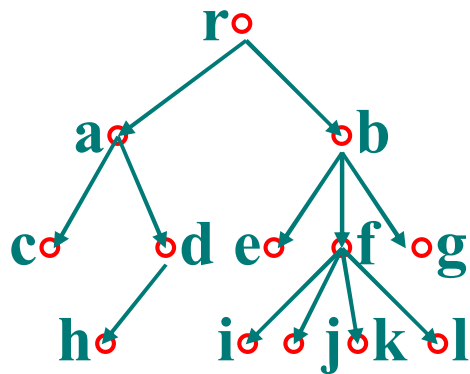
定理1 T是棵完全m叉树, 有t个叶结点, i个分支结点, 则 $(m-1)i=t-1$.

证明:T的所有结点的出度总和为 mi . 入度总和 $(i-1)+t$.
故 $mi=i-1+t$ 所以 $(m-1)i=t-1$

七. m叉有序树转化成二叉树

方法是:

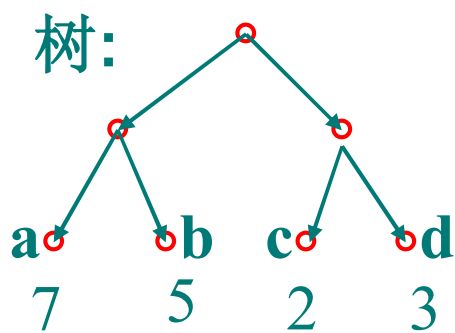
1. 每个结点保留左儿子结点, 剪掉右边其分支. 被剪掉的结点如下处理(重新嫁接).
2. 同一个层次的结点, 从左到右依次画出(被剪掉的结点 嫁接到它的哥哥结点上).



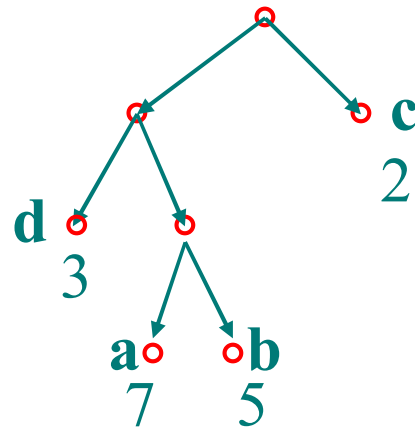
八. 最优树(哈夫曼树 Huffman)

1.带权二叉树的定义:设有一组权值: $w_1, w_2, w_3, \dots, w_m$, 不妨设 $w_1 \leq w_2 \leq w_3 \leq \dots \leq w_m$, 设有一棵二叉树有 m 片叶子, 分别带有权值 $w_1, w_2, w_3, \dots, w_m$, 称此树为带权二叉树.

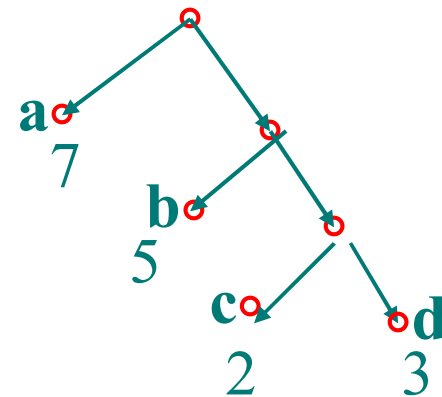
例如:下边是有叶结点 a, b, c, d , 分别带有权 $7, 5, 2, 3$ 的二叉树:



T_1



T_2

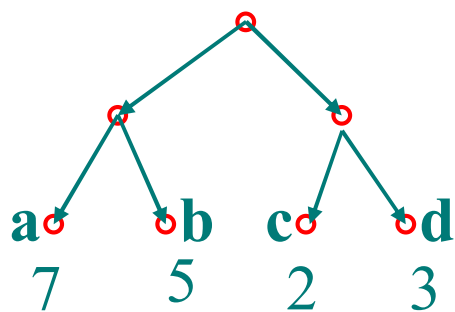


T_3

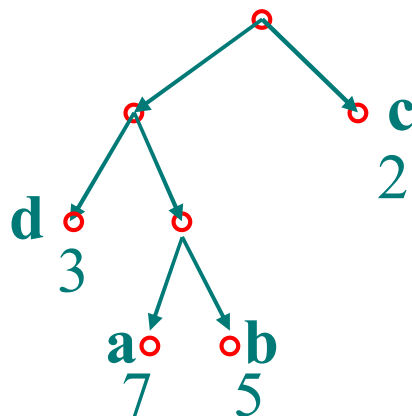
2. 带权树T的权W(T):

$$W(T) = \sum_{i=1}^m w_i L(w_i)$$

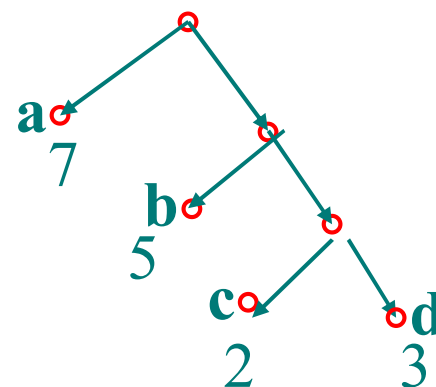
其中 $L(w_i)$ 是标有权 w_i 的叶结点的从根到该叶结点的路长.



T_1



T_2



T_3

3.最优树: 带权树中,权数最少的二叉树.

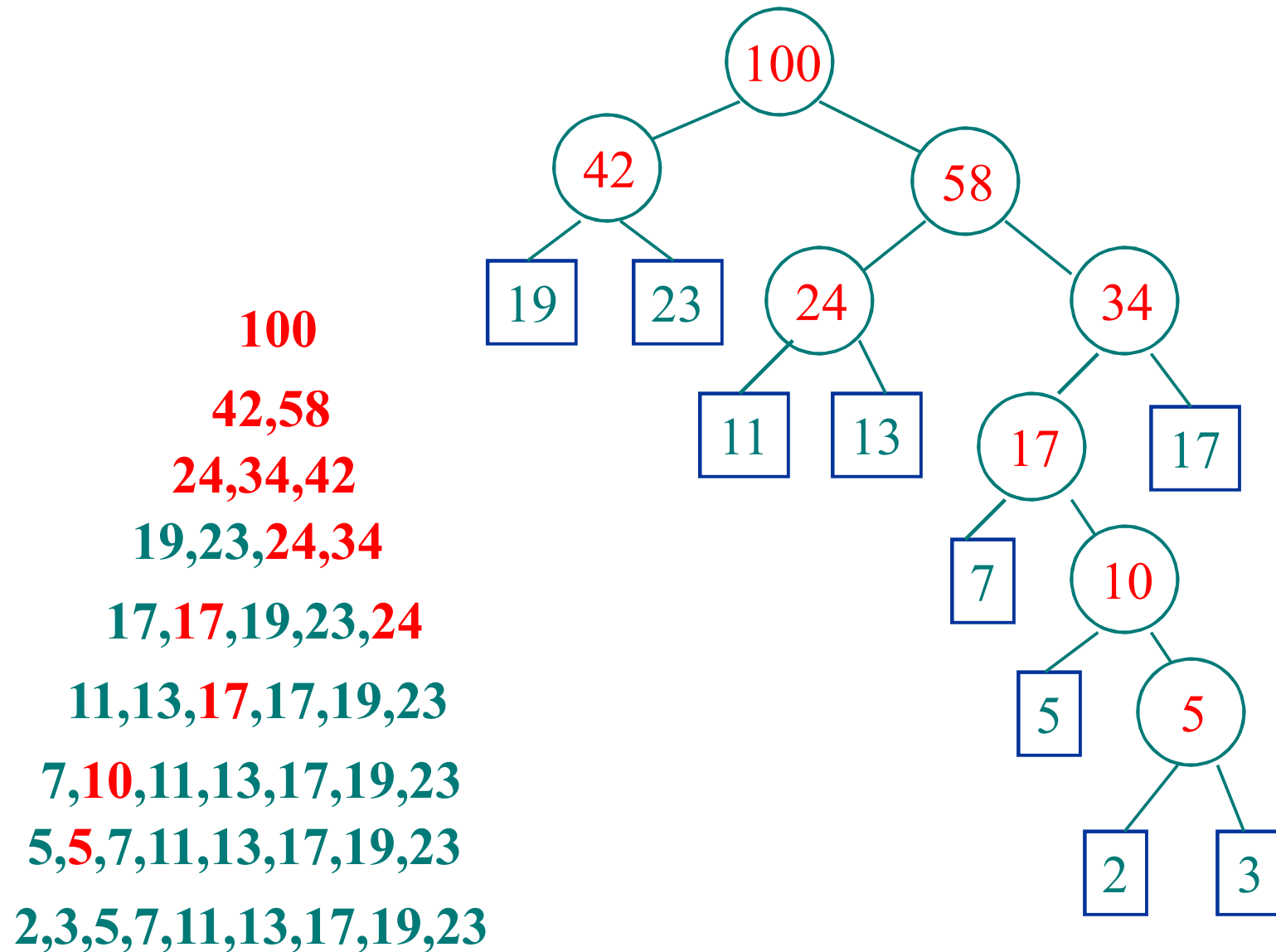
4.画最优树的算法---哈夫曼算法:

(1)先将权按照升序排序,设为 $w_1 \leq w_2 \leq w_3 \leq \dots \leq w_m$.

(2)以 w_1 和 w_2 为儿子结点,构造它们的父结点,且其权为 $w_1 + w_2$, 并从权的序列中去掉 w_1 和 w_2 。

(3) $w_1 + w_2$ 再与其余权一起排序,再从此队列中取出前面两个权值为儿子结点,同(2)的方法构造它们的父结点.
依此类推,直至最后. 即得到最优树.

例如,给定一组权:2,3,5,7,11,13,17,19,23. 构造一棵最优树.



5. 最优树的应用举例

(1) 用于程序设计

例如编写一个将百分分 a 转换成五计分的程序,如果这样:

if $a < 60$

then $b = \text{“不及格”}$

else if $a < 70$

then $b = \text{“及格”}$

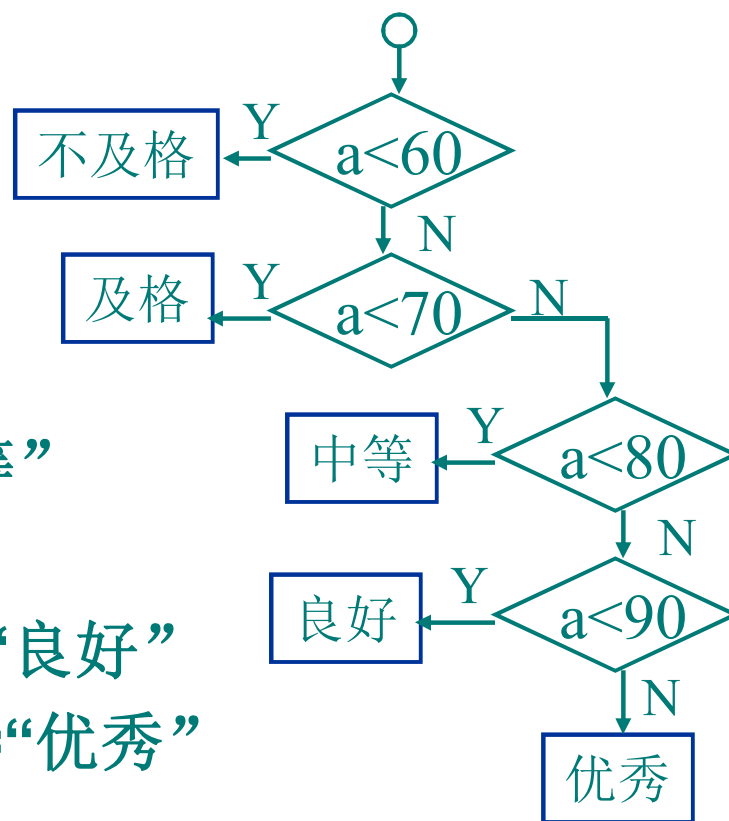
else if $a < 80$

then $b = \text{“中等”}$

else if $a < 90$

then $b = \text{“良好”}$

else $b = \text{“优秀”}$



上述程序是正确的,但不是最优的,.

衡量一个程序是否优化:

a)空间复杂性:一个是看它在运行时需要使用的存贮空间的大小,

b)时间复杂性:还要看它运行时间的长短.

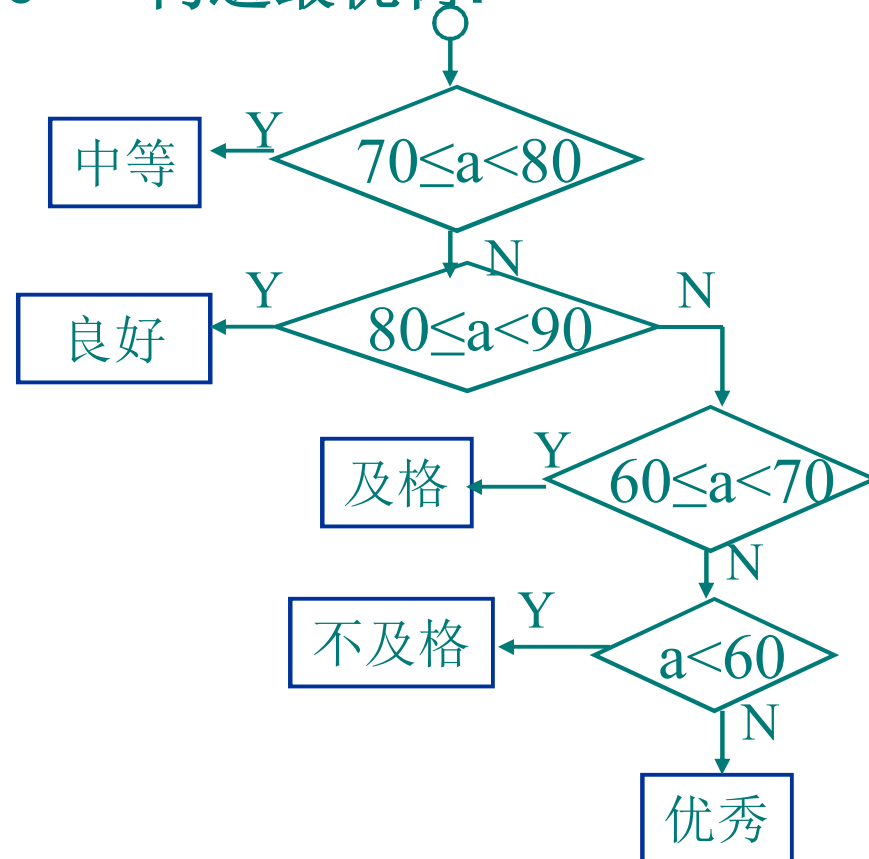
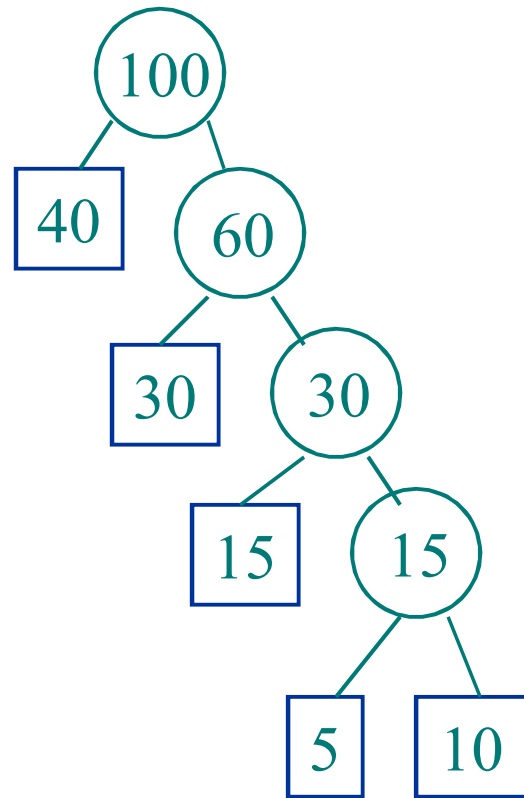
设分数的分布如下:

分数:	0-59	60-69	70-79	80-89	90-100
比例(%):	5	15	40	30	10

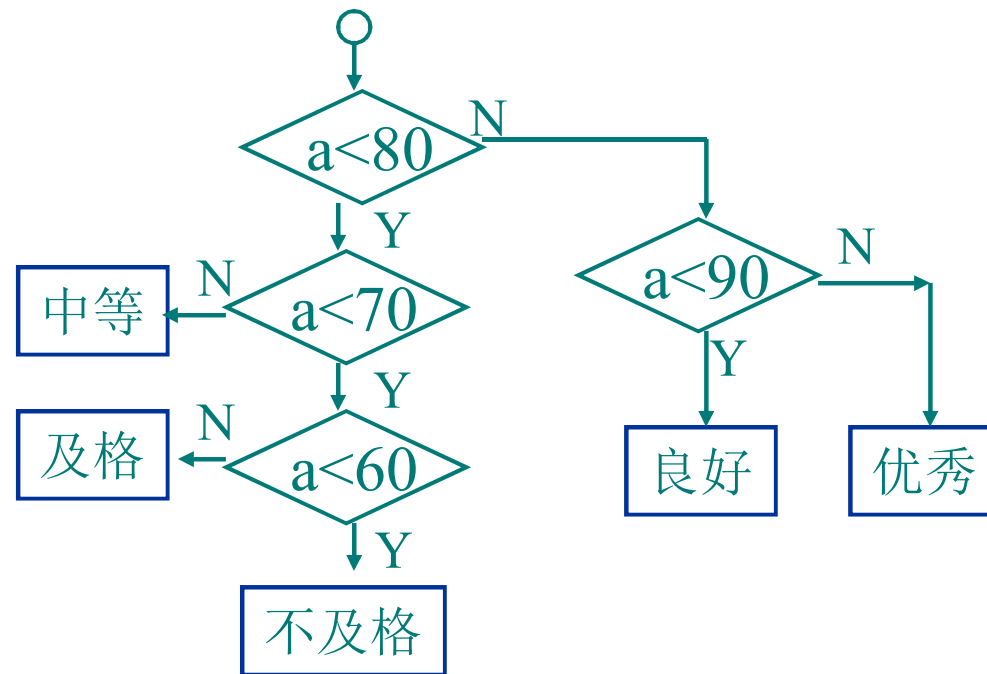
那么如何设计这个程序才合理呢?

分数:	0-59	60-69	70-79	80-89	90-100
比例(%):	5	15	40	30	10

设权序列为: 5,10,15,30,40 构造最优树:



为了使得判断框内只比较一次,流程图可以改成下面框图:



(2)前缀码(哈夫曼编码)

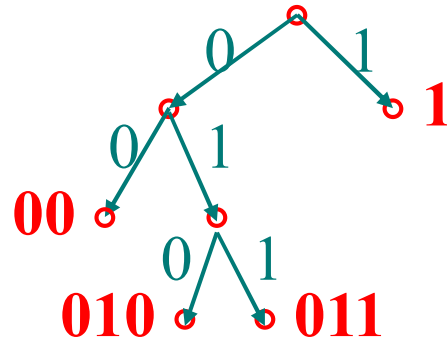
a) 问题的提出: 数据通讯时,需要将信息编码,即用二进制符号串表示信息.

前缀码

b)前缀码的定义:一个符号的编码不是另一个符号编码的前缀.

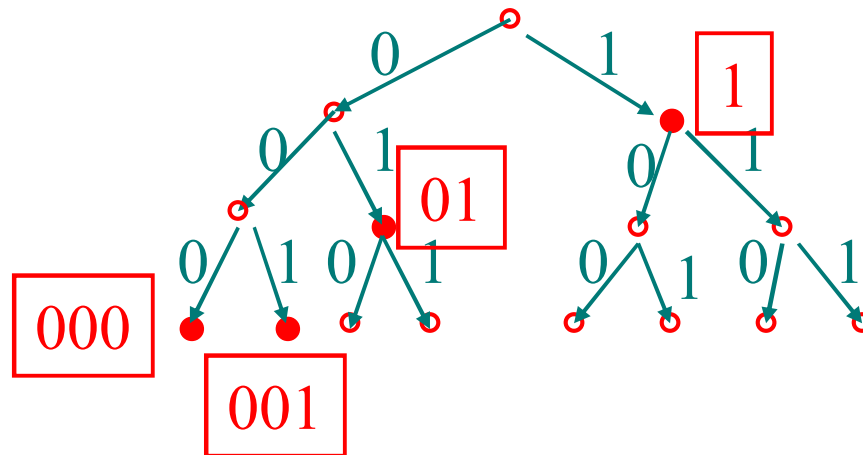
c)前缀码的设计:

每棵二叉树对应一组前缀码.



反之,任何一组前缀码也对应一棵二叉树.

例如有一组前缀码{1,01,001,000}, 可以画一棵高度为3的
二叉树,如图:



例：对报文“ABACCD A”设计编码.

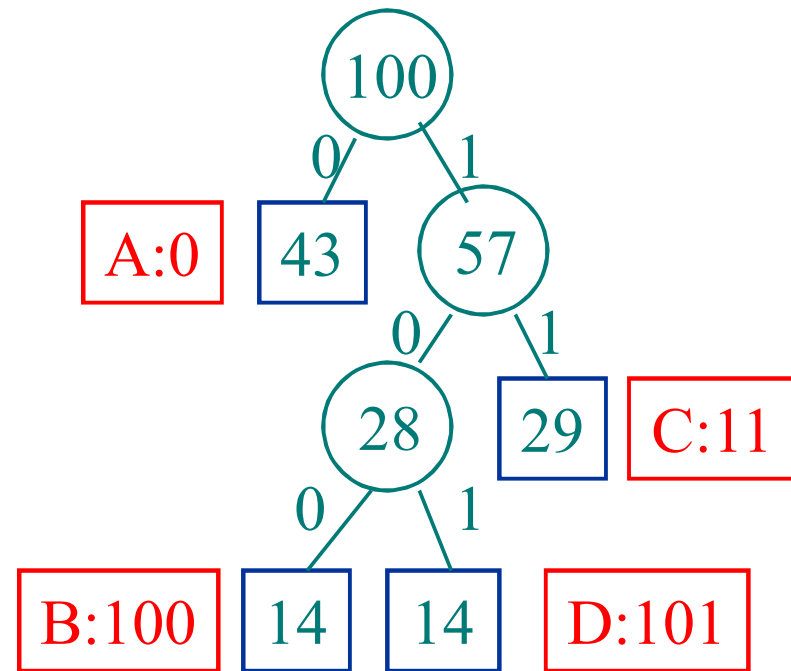
先计算各个符号在报文中出现的频率:A,B,C,D的频率分别是:43%,14%,29%,14% . 分别表示成权(43,14,29,14)

对权排序:14,14,29,43

画最优树:

编码是: A, B, C, D
0, 100, 11, 101.

原报文 “ABACCD A”译文:
“0100011111010”有13位.



那么在接收端如何译码呢?

就是将译文中符号从头读,

顺着这棵最优树,反复从根到叶找前缀码.

比如有如下报文 “01110110111100111100100”